Київський національний університет
Імені Тараса Шевченка.
Кафедра: Мережевих та інтернет-технологій.

Лабораторна робота №5

з дисципліни: Системне Програмування
На тему: “Динамічна ідентифікація типів”

Студента 4 курсу:
Групи МІТ-41
Мулико Володимира

Київ - 2025р.

1. Мета роботи

Вивчення принципів об'єктно-орієнтованого програмування, робота з властивостями, колекціями та відображенням складних структур даних у ієрархічних елементах керування.

2. Постановка задачі

1. Розробка графічного інтерфейсу:
   - Створити кросплатформний проєкт на базі фреймворку Avalonia UI (замість Windows Forms).
   - Додати на головну форму елемент керування TreeView для відображення ієрархічних даних.
   - Додати кнопку для ініціалізації об'єкта та виводу його характеристик.
2. Проєктування класу: Описати клас Movie (Кінофільм), забезпечивши наявність:
   - 4-х властивостей різних типів (наприклад: string, int, double та тип колекції List<string>).
   - 3-х методів для обробки даних об'єкта (наприклад: отримання короткої інформації, додавання елемента до колекції, логічна перевірка рейтингу).
   - 2-х конструкторів (за замовчуванням та з параметрами для ініціалізації всіх полів).
3. Реалізація алгоритму виводу: Описати метод, який виконує обхід властивостей об'єкта класу Movie та виводить їхній перелік (разом із типом та значенням) у TreeView. Колекції повинні відображатися як окремі вузли з вкладеними елементами.

Файл MainWindows.axaml

```
<Window xmlns="https://github.com/avaloniaui"

        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

        xmlns:local="clr-namespace:MovieApp"

        x:Class="MovieApp.MainWindow"

        x:DataType="local:MainWindow"
```

```
        Title="Лабораторна: №5"

        Width="400" Height="450">



    <StackPanel Margin="20" Spacing="10">

        <Button Content="Вивести дані у TreeView"

                Click="DisplayMovie_Click"

                HorizontalAlignment="Stretch"

                HorizontalContentAlignment="Center"/>



            <TreeView Name="MyTreeView" Height="300" BorderBrush="Gray"
BorderThickness="1">

            <TreeView.ItemTemplate>

                        <TreeDataTemplate ItemsSource="{Binding Nodes}"
x:DataType="local:TreeItem">

                    <TextBlock Text="{Binding Header}" />

                </TreeDataTemplate>

            </TreeView.ItemTemplate>

        </TreeView>

    </StackPanel>

</Window>
```

Файл [MainWindows.axaml.cs]

```
using Avalonia.Controls;

using Avalonia.Interactivity;

using System.Collections.Generic;

using System.Collections.ObjectModel;
```

```
namespace MovieApp
{
   public class TreeItem
   {
       public required string Header { get; set; }
          public ObservableCollection<TreeItem> Nodes { get; set; } =
new();
   }

   public partial class MainWindow : Window
   {
       public MainWindow()
       {
           InitializeComponent();
       }

        private void DisplayMovie_Click(object sender, RoutedEventArgs
e)
       {
                var myMovie = new Movie("Inception", 2010, 8.8, new
List<string> { "Sci-Fi", "Action", "Thriller" });
           var rootNodes = new ObservableCollection<TreeItem>();
               var movieRoot = new TreeItem { Header = $"Кінофільм:
{myMovie.Title}" };
```

```
                        AddPropertyToTree(movieRoot,  "Title",  "string",
myMovie.Title);

                           AddPropertyToTree(movieRoot,  "Year",  "int",
myMovie.ReleaseYear.ToString());

                       AddPropertyToTree(movieRoot,  "Rating",  "double",
myMovie.Rating.ToString());


               var genreNode = new TreeItem { Header = $"Genres (List,
Count: {myMovie.Genres.Count})" };

           foreach (var genre in myMovie.Genres)

           {

                genreNode.Nodes.Add(new TreeItem { Header = $"- {genre}"
});

           }

           movieRoot.Nodes.Add(genreNode);


           rootNodes.Add(movieRoot);




           MyTreeView.ItemsSource = rootNodes;

       }


          private void AddPropertyToTree(TreeItem parent, string name,
string type, string value)

       {

            parent.Nodes.Add(new TreeItem { Header = $"[{type}] {name}:
{value}" });

       }
```

```
    }

}
```

Файл Movie.cs

```
using System;

using System.Collections.Generic;


namespace MovieApp;


public class Movie

{

    // 4 властивості

    public string Title { get; set; } = "";

    public int ReleaseYear { get; set; }

    public double Rating { get; set; }

    public List<string> Genres { get; set; } = new();


    // 2 Конструктори

    public Movie() { }


     public Movie(string title, int year, double rating, List<string>
genres)

    {

        Title = title;

        ReleaseYear = year;
```

```
        Rating = rating;

        Genres = genres;

    }


    // 3 Методи

    public string GetShortInfo() => $"{Title} ({ReleaseYear})";


    public void AddGenre(string genre) => Genres.Add(genre);


    public bool IsHighlyRated() => Rating > 8.0;

}
```

Файл [Program.cs]

```
using Avalonia;

using System;


namespace MovieApp;


class Program

{

    // Точка входу в програму

    [STAThread]

    public static void Main(string[] args) => BuildAvaloniaApp()

        .StartWithClassicDesktopLifetime(args);
```

```
    // Налаштування Avalonia

    public static AppBuilder BuildAvaloniaApp()

        => AppBuilder.Configure<App>()

            .UsePlatformDetect()

            .LogToTrace();

}
```

Файлова система:

- **MovieApp.csproj**: Файл конфігурації проєкту .NET. Містить інформацію про версію SDK та залежності Avalonia.
- **Program.cs**: "Двигун" програми. Містить точку входу Main, яка ініціалізує графічну підсистему та запускає життєвий цикл застосунку.
- **App.axaml / App.axaml.cs**: Глобальні налаштування стилів (наприклад, FluentTheme) та визначення головного вікна, яке відкриється при старті.
- **Movie.cs**: Логічна модель (Back-end). Тут описана бізнес-логіка: властивості та методи нашого кінофільму.
- **MainWindow.axaml**: Декларативний опис інтерфейсу (Front-end) на мові XAML. Визначає розташування кнопки та TreeView.
- **MainWindow.axaml.cs**: "Code-behind". Тут міститься логіка обробки натискання кнопки та алгоритм трансформації даних об'єкта в дерево.

Опис моделі даних (Movie.cs)

Згідно з варіантом, ми реалізували 4 типи властивостей:

```
public string Title { get; set; } = "";

    public int ReleaseYear { get; set; }

    public double Rating { get; set; }
```

```
    public List<string> Genres { get; set; } = new();
```

Це дозволяє продемонструвати роботу з різними типами даних у пам'яті. Також додано методи, наприклад IsHighlyRated(), що відповідає вимогам про "наявність 3-х методів".

2. Побудова дерева (MainWindow.axaml.cs)

Для виводу в TreeView ми створили допоміжний клас TreeItem. Найголовніша частина логіки — це формування ієрархії:

```
var movieRoot = new TreeItem { Header = $"Кінофільм: {myMovie.Title}"
};
```

Тут ми створюємо "корінь" дерева. Далі ми викликаємо метод, який додає властивості як дочірні елементи:

```
private void AddPropertyToTree(TreeItem parent, string name, string
type, string value)

        {

            parent.Nodes.Add(new TreeItem { Header = $"[{type}] {name}:
{value}" });

        }
```

Цей метод автоматизує процес запису типу даних (.GetType().Name) та їх значення, що і було головною вимогою завдання.

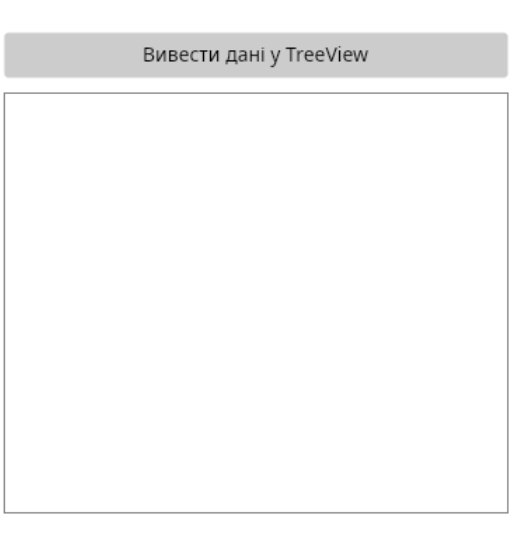
3. Інтерфейс та шаблон даних (MainWindow.axaml)

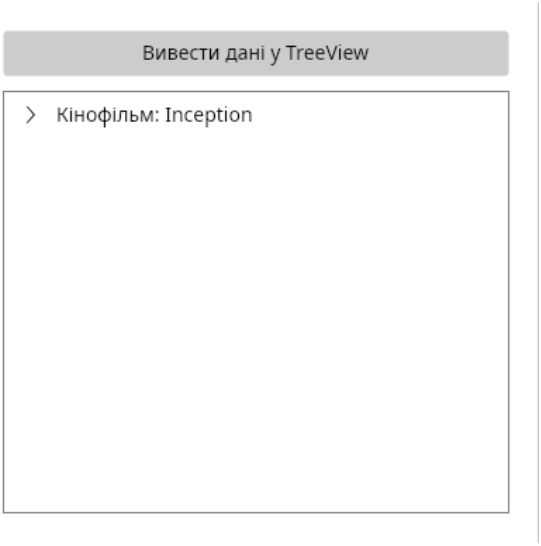
Щоб Avalonia розуміла, як малювати наше дерево, ми використали TreeDataTemplate:

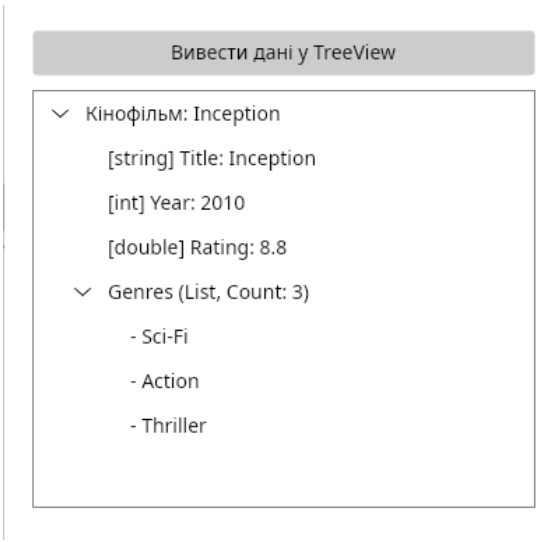
```
<TreeDataTemplate ItemsSource="{Binding Nodes}"
x:DataType="local:TreeItem">

                <TextBlock Text="{Binding Header}" />

            </TreeDataTemplate>
```

- **ItemsSource="{Binding Nodes}"**: Каже програмі, що якщо у вузла є список Nodes, його треба розгорнути як піддерево.
- **Text="{Binding Header}"**: Виводить текст заголовка, який ми сформували в коді (наприклад, "[int] Year: 2010").

Вивід програми:







Висновок:

Під час виконання лабораторної роботи було розроблено кросплатформний графічний застосунок мовою програмування C Sharp із використанням сучасного фреймворку Avalonia UI в операційній системі Arch Linux. Головною метою проєкту стала практична реалізація об’єктно-орієнтованої моделі даних та її візуалізація через ієрархічні структури керування. У межах завдання було спроєктовано та описано клас Movie, який містить чотири властивості різних типів, а саме рядкову назву фільму, цілочисельний рік випуску, дробовий рейтинг та динамічну колекцію жанрів у форматі списку рядків. Для забезпечення гнучкості ініціалізації об’єктів у коді реалізовано два конструктори, а для обробки даних — три методи, що дозволяють отримувати стислу інформацію про фільм, додавати нові жанри та виконувати логічну перевірку високого рейтингу. Особливу увагу приділено алгоритму виводу характеристик об’єкта в елемент керування TreeView, де для кожної властивості програма динамічно визначає її системний тип та поточне значення. Замість застарілого підходу Windows Forms у проєкті використано декларативну розмітку XAML та механізм скомпільованих прив’язок даних, що забезпечило коректне відображення вкладених вузлів для колекцій та високу продуктивність інтерфейсу. Програма складається з декількох ключових файлів, де Program.cs відповідає за запуск життєвого циклу застосунку, App.axaml визначає глобальні стилі, Movie.cs описує логічну модель фільму, а MainWindow реалізує графічну оболонку та логіку трансформації об’єкта в деревоподібний список. У результаті роботи було підтверджено працездатність обраної архітектури, успішно закріплено навички роботи з об’єктно-орієнтованим аналізом, інкапсуляцією та колекціями типу ObservableCollection, а розроблений застосунок повністю відповідає всім критеріям технічного завдання та демонструє стабільну роботу в середовищі Linux.